ICS 604
Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Homework assignment 2

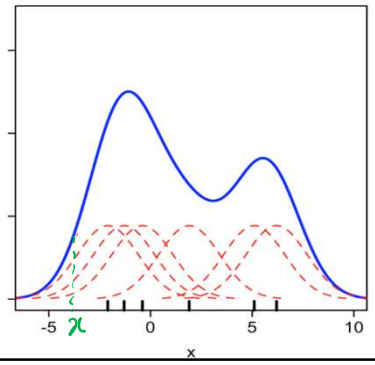
- Will be released after class.
- Due: **11:59 PM** on **Friday, March 13**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
  - Do not upload the data file.
  - Make sure to **rename your notebook file** as instructed.

Lecture 12

- KDE Bandwidth
- Introduction to Parameter Estimation
  - Poisson Distribution
- Parameter Estimation
  - Bootstrap Confidence Interval

Previously… Kernel density estimation (KDE)

- A method for estimating the probability density to infer characteristics of a population based on a finite data set.
- Kernel: a symmetric function used as a building block to estimate the probability density of a random variable.
- A kernel is centered at each data point.
  - The contribution of a data point to the estimated density at a given position is determined by the kernel values at that position.



Optimal kernel width

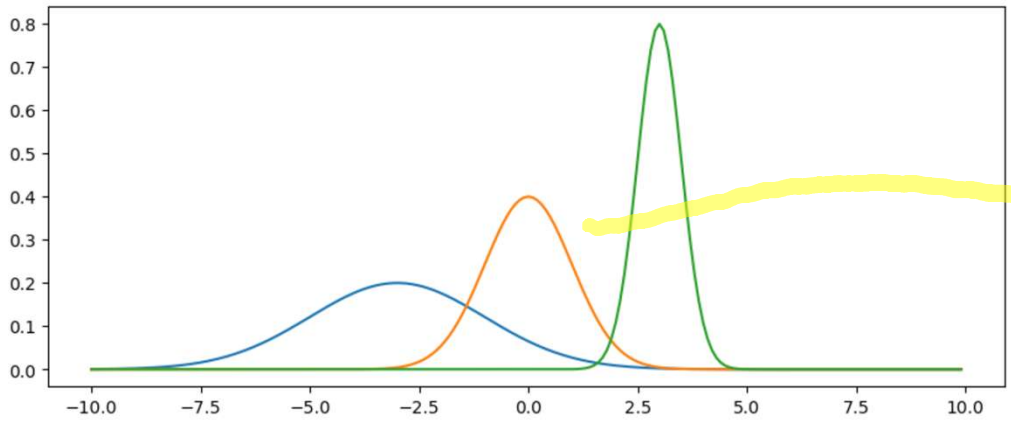
- Recall that the kernel is defined by its width.
  - Often called `bandwidth` in programming libraries.
  - This applies regardless of which kernel we choose (e.g., tophat, exponential, Gaussian, etc.)
- How do we define the appropriate kernel width?
  - In the previous example, we set `bw` to 0.16. Why did we choose this value?

Optimal kernel width (cont'd.)

- Recall that the scale determines the contributions of neighboring points.
  - Large width (high scale) accounts for distant neighbors.
  - Small width (lowl scale) discounts the contribution of distant neighbors.

```
fig = plt.figure(figsize=(10, 4))
kernel_1 = [sp.stats.norm.pdf(x, -3, 2) for x in np.arange(-10, 10, 0.1)]
kernel_2 = [sp.stats.norm.pdf(x, 0, 1) for x in np.arange(-10, 10, 0.1)]
kernel_3 = [sp.stats.norm.pdf(x, 3, 0.5) for x in np.arange(-10, 10, 0.1)]

plt.plot(np.arange(-10,10, 0.1), kernel_1)
plt.plot(np.arange(-10,10, 0.1), kernel_2)
plt.plot(np.arange(-10,10, 0.1), kernel_3);
```

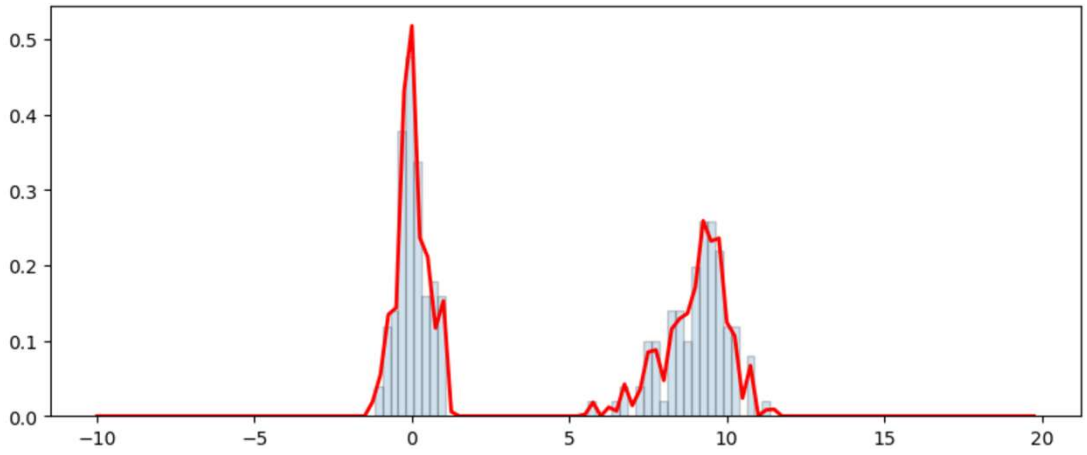


Optimal kernel width (cont'd.)

- It is essential to choose an appropriate bandwidth, as values that are too small or too large will not be useful.
  - If bandwidth is **too small**, the KDE is said to **under-smooth**.
    - Contributions to nearby points is small, which leads to a jagged curve.
    - This also referred to in machine learning as **overfitting**.

```
kde = sp.stats.gaussian_kde(all_data, bw_method=0.02)
densities = kde.evaluate(x_values)

fig = plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=50, density=True, edgecolor="k", linewidth=1, alpha=0.2)
plt.plot(x_values, densities, lw=2, color='r');
```



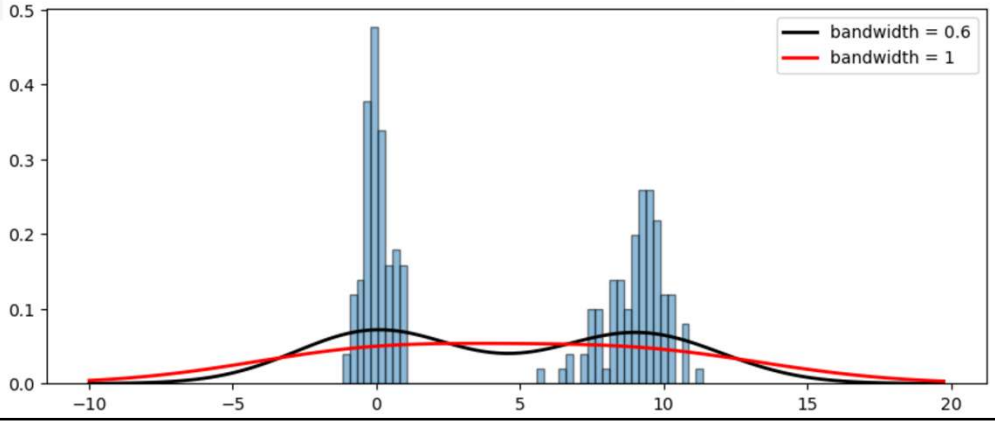
Optimal kernel width (cont'd.)

- If the bandwidth is **too large**, the KDE is said to **over-smooth**.
  - Contributions of nearby points is large, which leads to a flattened curve.
  - This also referred to in machine learning as **underfitting**.

```
kde = sp.stats.gaussian_kde(all_data, bw_method=0.6)
densities = kde.evaluate(x_values)

kde = sp.stats.gaussian_kde(all_data, bw_method=1)
densities_2 = kde.evaluate(x_values)

plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=50, density=True, edgecolor="k", linewidth=1, alpha=0.5)
plt.plot(x_values, densities, lw=2, color='black', label="bandwidth = 0.6")
plt.plot(x_values, densities_2, lw=2, color='red', label="bandwidth = 1")
plt.legend();
```



How do we choose the correct value of the bandwidth?

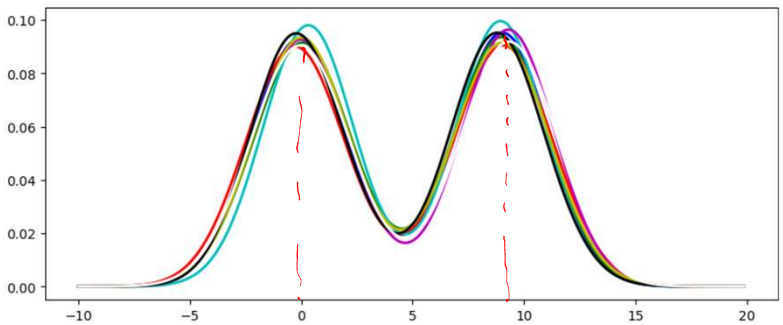
- Recall that we are trying to estimate a probability density for the population in the presence of a small dataset.
  - We want the bandwidth to be chosen such that the estimated density is as close to the population pdf as possible.
  - We want a bandwidth that is narrow enough to retain all relevant details, but wide enough so that the resulting curve is not too wiggly.
    - This is a recurring problem in data analysis; **Bias-Variance Tradeoff**

Illustrating the bias-variance tradeoff

- To illustrate, we are going to generate multiple datasets from a population containing two distributions.
  - $X \sim \mathcal{N}(0, 1)$
  - $Y \sim \mathcal{N}(9, 1)$
- All the dataset come from the same population, so we expect their estimated probability densities to be as close as possible.

```
plt.figure(figsize=(10, 4))
x_values = np.arange(-10, 20, 0.1)
x_mean, x_scale = 0, 1
y_mean, y_scale = 9, 1

colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
for i in range(len(colors)):
    x_data = np.random.normal(x_mean, x_scale, 50)
    y_data = np.random.normal(y_mean, y_scale, 50)
    data = np.concatenate([x_data, y_data])
    kde = sp.stats.gaussian_kde(data, bw_method=0.4)
    densities = kde.evaluate(x_values)
    plt.plot(x_values, densities, lw=2, color=colors[i])
```



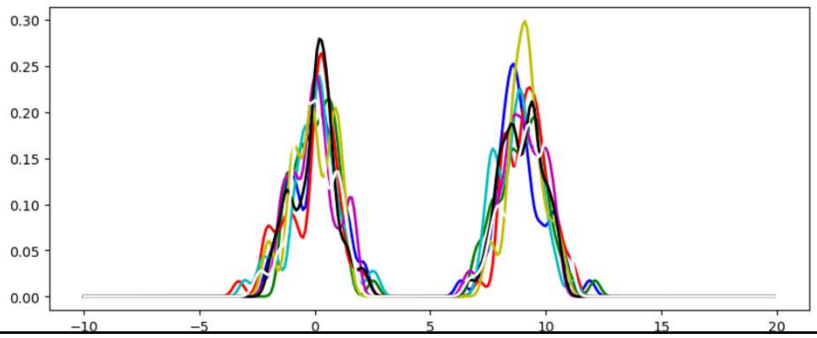
Model variance

- Variance in this context is the variability in the model's output that is due to small fluctuations in the data.
- The model with a high variance:
  - Pays close attention to the actual data and does not generalize to data it hasn't seen before.

    ⇒ Such models perform very well on data used to learn the model (training) but have high error rates on new (test) data.

```
plt.figure(figsize=(10, 4))
x_values = np.arange(-10, 20, 0.1)
x_mean, x_scale = 0, 1
y_mean, y_scale = 9, 1

colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
for i in range(len(colors)):
    x_data = np.random.normal(x_mean, x_scale, 50)
    y_data = np.random.normal(y_mean, y_scale, 50)
    data = np.concatenate([x_data, y_data])
    kde = sp.stats.gaussian_kde(data, bw_method=0.05)
    densities = kde.evaluate(x_values)
    plt.plot(x_values, densities, lw=2, color=colors[i])
```

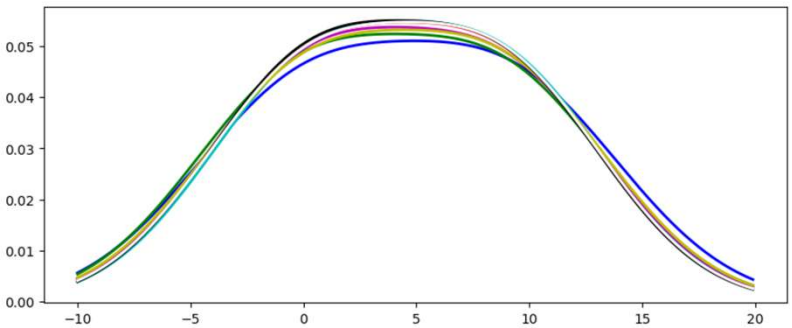


Model bias

- Bias in this context is the difference between the predicted and actual densities.
  - High-bias models oversimplify the estimated density.
  - Data contribution to the model is minimized. So, regardless of what the data says, the model will still look relatively the same.

```
plt.figure(figsize=(10, 4))
x_values = np.arange(-10, 20, 0.1)
x_mean, x_scale = 0, 1
y_mean, y_scale = 9, 1

colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
for i in range(len(colors)):
    x_data = np.random.normal(x_mean, x_scale, 50)
    y_data = np.random.normal(y_mean, y_scale, 50)
    data = np.concatenate([x_data, y_data])
    kde = sp.stats.gaussian_kde(data, bw_method=1)
    densities = kde.evaluate(x_values)
    plt.plot(x_values, densities, lw=2, color=colors[i])
```

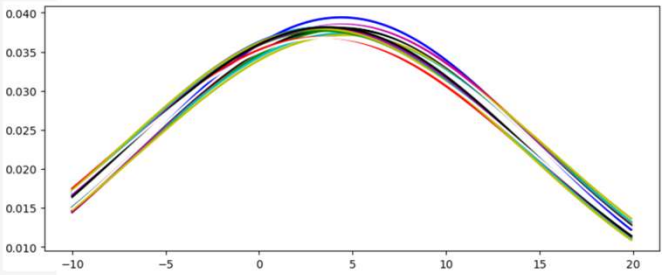


```
plt.figure(figsize=(10 ,4))
x_values = np.arange(-10, 20, 0.1)
x_mean, x_scale = 0, 1
y_mean, y_scale = 9, 1

colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k', 'w']
for i in range(len(colors)):
    x_data = np.random.normal(x_mean, x_scale, 50)
    y_data = np.random.normal(y_mean, y_scale, 50)
    data = np.concatenate([x_data, y_data])
    kde = sp.stats.gaussian_kde(data, bw_method=2)
    densities = kde.evaluate(x_values)
    plt.plot(x_values, densities, lw=2, color=colors[i])

x_mean, x_scale = -1, 1
y_mean, y_scale = 8, 1

for i in range(len(colors)):
    x_data = np.random.normal(x_mean, x_scale, 50)
    y_data = np.random.normal(y_mean, y_scale, 50)
    data = np.concatenate([x_data, y_data])
    kde = sp.stats.gaussian_kde(data, bw_method=2)
    densities = kde.evaluate(x_values)
    plt.plot(x_values, densities, lw=2, color=colors[i])
```



Model variance and bias revisited

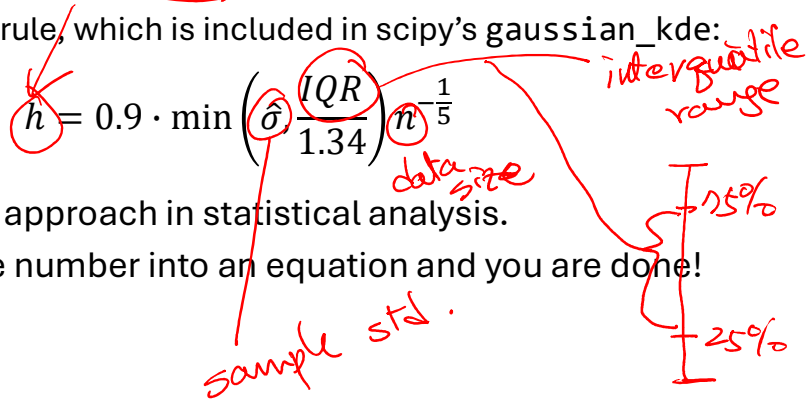
- Based on the above, we can conclude that:
  - Models with high variance, as the name implies, tend to generate a greater variance in their predictions for inferred values.
  - Models with high bias tend to generate greater constant biased model that do not fit the data well.
- What we want is to find the bandwidth that best fits our data.
  - Yields a model with minimal bias and variance.
- We can assure that the bandwidth fits the data by leaving some data out and finding the bandwidth that maximizes our likelihood.

Selecting the kernel bandwidth: Theoretical

- Theoretical versus empirical solutions to tackle this problem.
- Theoretical: optimal bandwidth is estimated using an equation.
  - Ex. Silverman's rule, which is included in scipy's `gaussian_kde`:

$$\hat{h} = 0.9 \cdot \min\left(\hat{\sigma}, \frac{IQR}{1.34}\right) n^{-\frac{1}{5}}$$

- This is the typical approach in statistical analysis.
- Very fast: plug the number into an equation and you are done!



Selecting the kernel bandwidth: Empirical

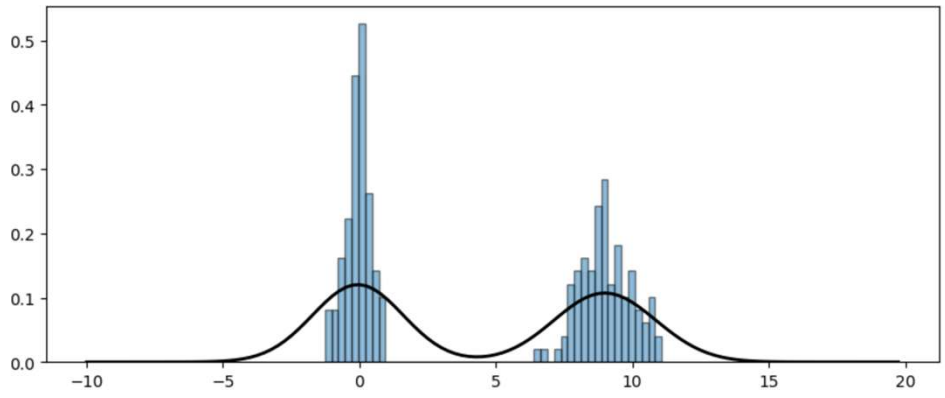
- An empirical approach that relies on the data.
  - Choose a subset of the data from the KDE and see how well the KDE fits the remaining data.
  - Iterate until you find the best KDE.
    - This approach is called cross-validation.
- This approach will be covered in the machine learning lectures later.

Selecting the kernel bandwidth: Practical

- For KDE, it is often sufficient to start with the default bandwidth suggested by the statistical method and then adjust it slightly to produce the most intuitive visualization of the point distributions.
- Here, we can select a bandwidth that aligns with our prior knowledge of the data.
  - We know the data comes from two distributions.

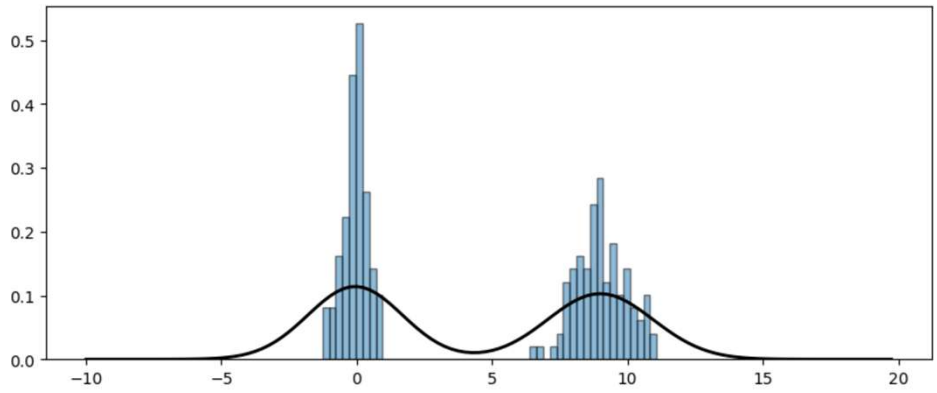
```
x_values = np.arange(-10, 20, 0.25)
kde = sp.stats.gaussian_kde(all_data)  # bw_method='scott' by default
densities = kde.evaluate(x_values)

plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=50, edgecolor="k", linewidth=1, density=True, alpha=0.5)
plt.plot(x_values, densities, lw=2, color='black');
```



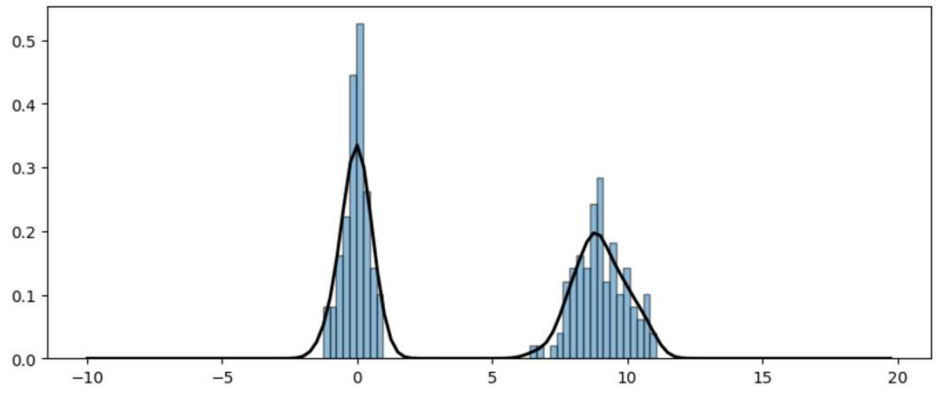
```
x_values = np.arange(-10, 20, 0.25)
kde = sp.stats.gaussian_kde(all_data, bw_method='silverman')
densities = kde.evaluate(x_values)

plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=50, edgecolor="k", linewidth=1, density=True, alpha=0.5)
plt.plot(x_values, densities, lw=2, color='black');
```




```
x_values = np.arange(-10, 20, 0.25)
kde = sp.stats.gaussian_kde(all_data, bw_method=0.09)
densities = kde.evaluate(x_values)

plt.figure(figsize=(10, 4))
plt.hist(all_data, bins=50, edgecolor="k", linewidth=1, density=True, alpha=0.5)
plt.plot(x_values, densities, lw=2, color='black');
```



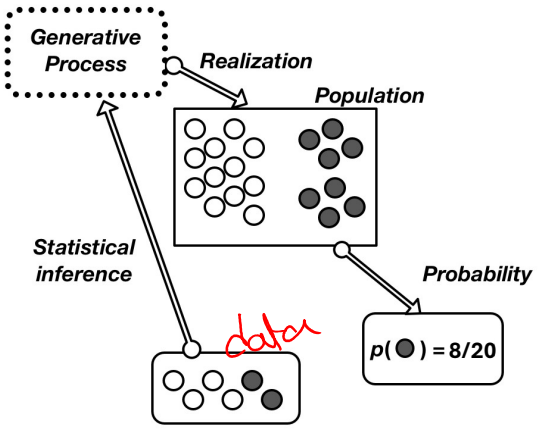
Introduction to Parameter Estimation

Generative modeling

- Generative models are expressive representations of the data generating process.
  - A generative model describes how likely a given example is to be generated.
  - Having access to a distribution's parameters is very useful to answer the different types of questions we have been interested in so far:
    1. Computing probabilities, expectations, and variances of distributions
    2. Performing hypothesis testing
    3. Making probabilistic predictions
    4. Simulating new data from the estimated distribution
- Most often we don't have access to parameters that generated the data.
- Therefore, we need to estimate those parameters from a small sample.

Statistical inference

- Contrary to generative models, to estimate a distribution's parameters we work from the data upwards.
- This upward–reasoning step is called *statistical inference*.



Statistical inference (cont'd.)

- We often work with a small sample drawn from a large population.
  - **Population**: the complete set of individuals or outcomes of interest.
  - **Sample**: a subset of the population that we can observe and measure.
- Why sample?
  - Observing the entire population is usually impractical or impossible.
  - Sample statistics (e.g., sample mean) are used as proxies for unknown parameters of the population, assuming the sample is representative.
  - A sample provides information about the underlying probability distribution.
  - Enables estimation of parameters, probabilities, expectations, and other quantities of interest.

Parameter estimation

- Given a dataset, we are most often interested in estimating the parameter(s) of the distribution that generated it.
- E.g., we randomly count the number of observed moving traffic citations within a 5-mile radius of UH Manoa over 90 days, spread across 12 months.
  - We want to even out sampling to avoid any bias.
- The number of citations should follow some probability distribution specific to count data.
  - This is a Poisson distribution.

Parameter estimation (cont'd.)

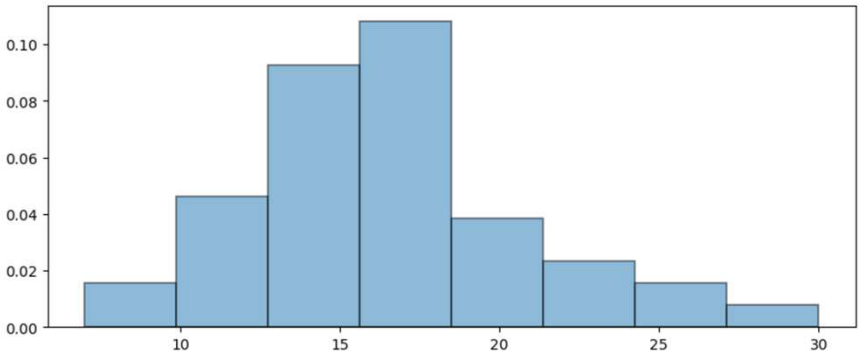
- How do we estimate the parameter of the Poisson distribution from our observed data?
  - We will answer this question using three different approaches:
    1. Bootstrap Confidence Intervals
    2. Maximum Likelihood
    3. Bayesian Framework

```
citations_data = pd.read_csv("data/citations_counts.tsv", index_col="Day")
citations_data.head()
```

Day	Counts
0	27
1	18
2	30
3	13
4	20

```
plt.figure(figsize=(10, 4))

_ = plt.hist(citations_data["Counts"], bins=8, density=True,
             edgecolor='black', linewidth=1.2, alpha=0.5)
```



The Poisson distribution

- Used in experiments that model the numbers of events occurring in a fixed period of time or a fixed area of space.
- For example:
  - Number of daily accidents on the H1
  - Number of people who enter a grocery store every hour
  - Number of childbirths in Hawaii every day
  - Number of chocolate chips in a cookie
  - Etc.

Properties of the Poisson distribution

- Poisson distribution has only one parameter that determines its shape and location.
  - The parameter is referred to as λ.
- A random variable X has a Poisson distribution, written $X \sim Poisson(\lambda)$, if:
  - X is the number of events occurrence in a given time period or space area.
    - Countably infinite non-negative values [0, 1, …].
  - The events are independent
    - The occurrence of events does not increase the likelihood of other events occurring.
  - The mean and the variance of a Poisson distribution are represented using the same parameter λ.
- The pmf of a Poisson distribution is: $P(X = k) = \frac{e^{-\lambda}\lambda^{k}}{k!}$

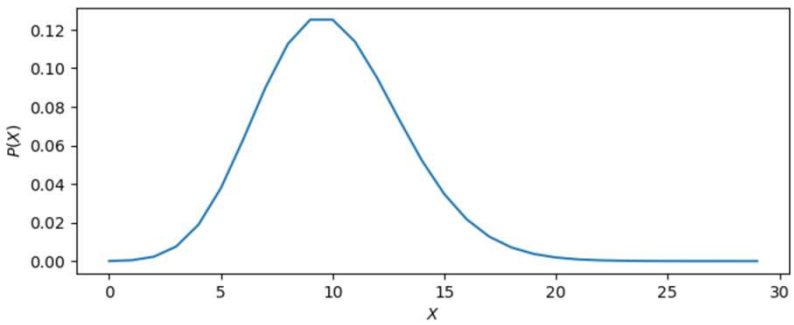
Example

- Assume that the customers visiting a store per hour follows a Poisson distribution with $\lambda = 10$. Then, the probability distribution of the random variable X representing the number of customers in the store is:

```
from scipy.stats import poisson

x = np.arange(0, 30, 1)
pmfs_x = poisson.pmf(x, 10)

plt.figure(figsize=(8, 3))
plt.plot(x, pmfs_x)
plt.xlabel("$X$")
_ = plt.ylabel("$P(X)$")
```



Example (cont'd.)

- The probability that x, for $x \in \{0, 5, 10, 15, 20, 30, 50\}$, customers visit the store tomorrow between 3 PM and 4 PM is:

```
for x in [0, 5, 10, 15, 20, 30, 50]:
    print(f"The pmf of x = {x:2d} is {poisson.pmf(x, 10):1.20f}")
```

```
The pmf of x =  0 is 0.00004539992976248485
The pmf of x =  5 is 0.03783327480207079180
The pmf of x = 10 is 0.12511003572113371662
The pmf of x = 15 is 0.03471806963068424512
The pmf of x = 20 is 0.00186608131399877415
The pmf of x = 30 is 0.00000017115717355368
The pmf of x = 50 is 0.00000000000000000015
```

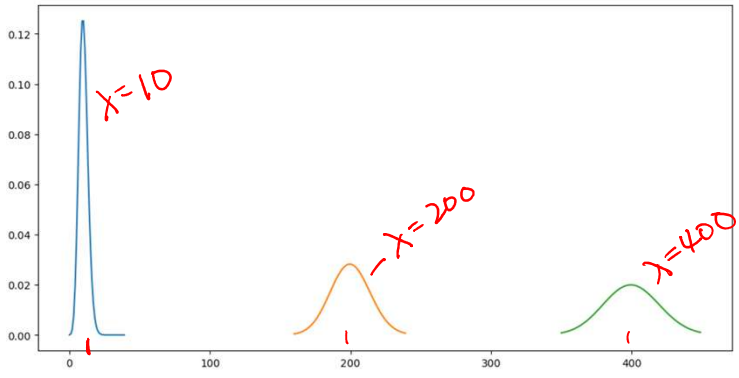
Link between the mean and the variance

- As mentioned, the mean and variance are both modeled with the same parameter λ.
  - What is the significance of this statement?

```
x = np.arange(0, 40, 1)
pmfs_x = poisson.pmf(x, 10)
plt.plot(x, pmfs_x)

x = np.arange(160, 240, 1)
pmfs_x = poisson.pmf(x, 200)
plt.plot(x, pmfs_x)

x = np.arange(350, 450, 1)
pmfs_x = poisson.pmf(x, 400)
_ = plt.plot(x, pmfs_x)
```



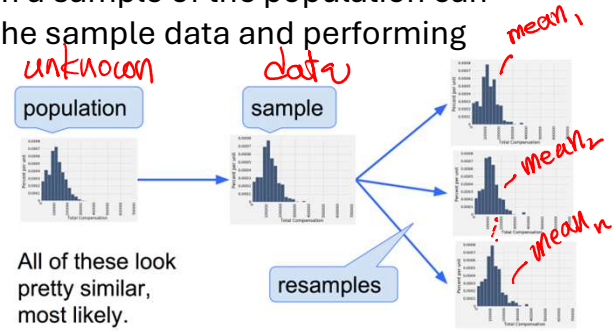
Notes about the Poisson distribution

- For values of $\lambda > 20$, the Poisson distribution can be approximated as Gaussian with $\mu = \lambda$ and $\sigma = \sqrt{\lambda}$.
  - This can be shown mathematically, and this simplification sometimes makes the math easy.
- Not all count data is Poisson.
  - Data where the mean is not equal to the variance is not Poisson.
    - Occurs in over-dispersed data when external factors, such as noise, contribute to the observed variance.
      - Negative Binomial is often used with such data.

Parameter Estimation: Bootstrap Confidence Interval

Bootstrapping

- Bootstrapping consists of sampling with replacement from observed dataset.
- *Basic idea*:
  - Inference about a population from a sample of the population can be modeled by resampling from the sample data and performing inference on the resampled sample.
- In bootstrap resamples, the 'population' is in fact the sample.



How bootstrapping is used

- The bootstrap data shows us the extent of sampling variability.
- Common uses:
  - Approximate the sampling distribution of a statistic (e.g., mean, median, variance)
  - Estimate standard errors → variability in an estimate
  - Construct confidence intervals
  - Assess stability of model estimates

How bootstrapping is used (cont'd.)

- Procedure
  1. Draw many resamples (with replacement) from the original dataset.
  2. Compute the statistic of interest for each resample.
  3. Examine the distribution of the computed statistics.
- Key Idea
  - The original sample serves as a stand-in for the population.
  - Repeated resampling approximates the variability we would observe from repeated sampling of the true population.

Using bootstrap to estimate the population mean

- In the following example, we draw a sample of size 100 from a normal distribution with $\mu = 50$ and $\sigma = 10$.

```
np.random.seed(22)
data = np.random.normal(50, 10, 100)
```

- Are 100 records sufficient to obtain a good measure of the population mean using bootstrap?
- What is the population's range of possible means that can be estimated from samples of this size?

```
# Given the following

np.random.seed(22)

data = np.random.normal(50, 10, 100)
data.mean()
```

```
49.46324599387749
```

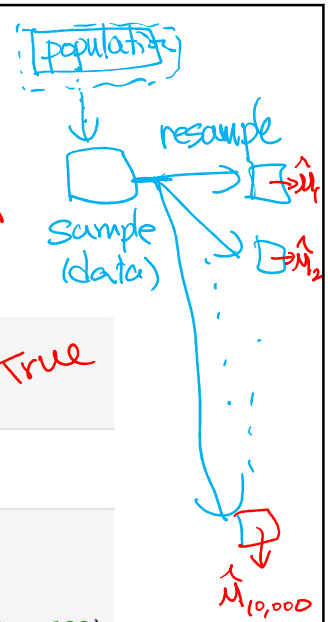
```
# One bootstrap resample - Number of distinct values

len(set(np.random.choice(data, 100)))
```

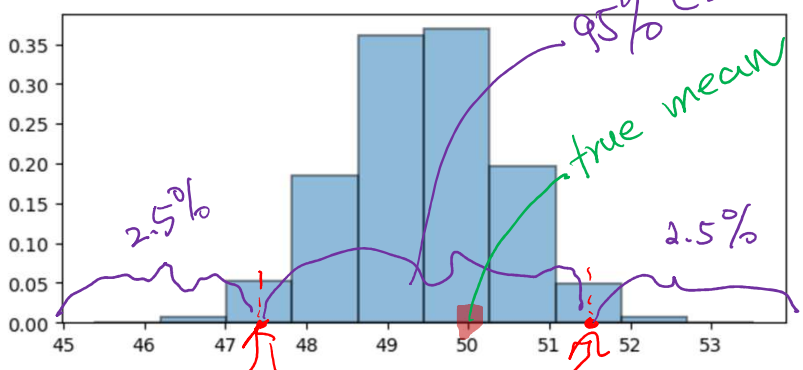
```
65
```

```
bootstrap_means = []

for i in range(10_000):
    returns_data_100_bootstrap = np.random.choice(data, 100)
    bootstrap_mean = returns_data_100_bootstrap.mean()
    bootstrap_means.append(bootstrap_mean)
```




```
plt.figure(figsize=(7, 3))
plt.hist(bootstrap_means, density=True, edgecolor='black', linewidth=1.2, alpha=0.5)
plt.show()
```




```
np.percentile(bootstrap_means, (2.5, 97.5))
```

```
array([47.49032939, 51.35670114])
```

Interpreting the bootstrap values

- The bootstrap data shows us the extent of sampling variability.
- The 95% confidence interval contains our population mean.
  - Values within 95% confidence interval are not a fluke.
  - We cannot discredit the fact that any of those values are possible means of the population.
- Conclusion:
  - Even with only 100 samples, we can observe that population mean is included in the interval [47.49, 51.36].

How confident are we in the bootstrap confidence interval?

- The estimate 95% confidence interval captures the population parameter. Was that a fluke?
- To see how frequently the interval contains the parameter, we have to run the entire process over and over again.
- We will repeat the following process a number of times (say 100 times):
  - Generate a new data sample of size 100 from the population.
  - Generate 10,000 bootstrap resamples from the data sample.
  - Compute the 95% confidence interval for the mean.
- We will end up with 100 intervals, and count how many of them contain the population mean.
  - How many of these 95% confidence intervals will include our mean?

```python
def compute_conf_interval(data, nb_bootstrap_iters = 10_000):
    bootstrap_means = []
    for i in range(nb_bootstrap_iters):
        bootstrap_sample = np.random.choice(data, 100, replace=True)
        bootstrap_means.append(bootstrap_sample.mean())
    return np.percentile(bootstrap_means, (2.5, 97.5))

lower_bound = []
upper_bound = []
for i in range(100):
    data = np.random.normal(50, 10, 100)
    conf_interval = compute_conf_interval(data)
    lower_bound.append(conf_interval[0])
    upper_bound.append(conf_interval[1])
```

simulation

new sample of 100

```python
conf_ints_95 = pd.DataFrame({"lower": lower_bound, "upper": upper_bound})
conf_ints_95.head()
```

	lower	upper
0	48.652341	52.444057
1	48.368890	52.473363
2	48.343475	52.276448
3	46.143426	50.403539
4	48.419776	52.061113

```python
conf_ints_95.shape
```

```
(100, 2)
```

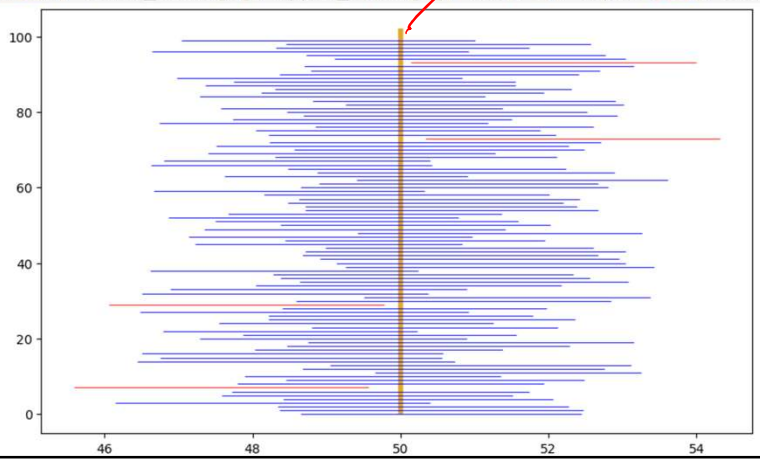
```python
sum((conf_ints_95["lower"] < 50) & (conf_ints_95["upper"] > 50))
```

```
96
```

← # of 95% CIs containing the true mean.

```
plt.figure(figsize=(10, 6))
plt.vlines(50, 0, 102, color="#e2a829", linewidth=4)

for i in range(100):
    c = "blue"
    if lower_bound[i] > 50 or upper_bound[i] < 50:
        c = "red"
    plt.hlines(i, lower_bound[i], upper_bound[i], color=c, alpha=0.85, linewidth=0.75)
```



How confident are we in the bootstrap confidence interval? (cont'd.)

- If an interval doesn't cover the parameter, it's a failure.
  - There are very few of them in this case.
- Any method based on sampling has the possibility of being off.
  - If we have a 95% confidence interval, we should expect to be wrong 5% of the time.
    - Statistical expectation over a long run of experiments.
    - If you were to construct confidence intervals an extremely large number of times under the same conditions, 95% of those intervals would contain the true parameter value, leaving 5% that do not.
- The beauty of methods based on random sampling is that we can quantify how often they are likely to be off.